7-3. Spring JPA 구성 및 웹 기능 추가

Spring Boot + JPA 보안 관리

1. pom.xml 설정

- 아래 코드를 pom.xml의 <dependencies> 부분에 추가하고 저장

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xs
i="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 h
ttps://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.3</version>
        <relativePath/> <!-- lookup parent from repository
-->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>demo</name>
    <description>Demo project for Spring Boot</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
```

```
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Thymeleaf -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifact
Id>
    </dependency>

    <!-- JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactI
d>
    </dependency>

    <!-- DB (MySQL, H2 for test) -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Lombok -->
    <dependency>
```

```
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
    <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactI
d>
            <scope>test</scope>
        </dependency>

</dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artif
actId>
            </plugin>
        </plugins>
    </build>

</project>
```

2. application-prod.properties

```
spring.application.name=demo
server.port=8081

spring.datasource.url=jdbc:mysql://{Mysql Endpoint}:3306/de
```

```
mo?useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.username=admin
spring.datasource.password=cicd1234!

spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.diale
ct.MySQLDialect
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

---

3. Spring Boot 코드

Member Entity

```
package com.example.demo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Bui
lder
public class Member {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String name;
```

```
    @Column(unique = true, nullable = false, length = 100)
    private String email;
}
```

Repository

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepositor
y;

public interface MemberRepository extends JpaRepository<Mem
ber, Long> {}
```

Service

```
package com.example.demo;

import java.util.List;

import org.springframework.stereotype.Service;

import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class MemberService {
    private final MemberRepository memberRepository;
    public Member save(Member member) { return memberReposi
tory.save(member); }
    public List<Member> findAll() { return memberRepositor
y.findAll(); }
}
```

Controller

```
package com.example.demo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribu
te;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMappi
ng;

import lombok.RequiredArgsConstructor;

@Controller
@RequiredArgsConstructor
@RequestMapping("/members")
public class MemberController {
    private final MemberService memberService;

    @GetMapping
    public String list(Model model) {
        model.addAttribute("members", memberService.findAll
());
        return "members/list";
    }

    @GetMapping("/new")
    public String createForm(Model model) {
        model.addAttribute("member", new Member());
        return "members/createForm";
    }

    @PostMapping
    public String create(@ModelAttribute Member member) {
        memberService.save(member);
        return "redirect:/members";
    }
```

```
}
```

4. Thymeleaf 템플릿

members/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>회원 목록</title>
    <style>
        table {
            width: 60%;
            border-collapse: collapse;
            margin-top: 20px;
        }
        th, td {
            border: 1px solid #ddd;
            padding: 8px;
            text-align: left;
        }
        th {
            background-color: #f2f2f2;
        }
        a {
            display: inline-block;
            margin-bottom: 10px;
        }
    </style>
</head>
<body>
    <h1>회원 목록</h1>

    <a href="/members/new">회원 등록</a>
```

```
    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>이름</th>
                <th>Email</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="m : ${members}">
                <td th:text="${m.id}"></td>
                <td th:text="${m.name}"></td>
                <td th:text="${m.email}"></td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

members/createForm.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>회원 등록</title>
    <style>
        form {
            display: flex;
            flex-direction: column;
            width: 300px;
            margin-top: 20px;
        }
        label {
            margin-bottom: 10px;
        }
        input {
            padding: 5px;
```

```
            margin-top: 2px;
        }
        button {
            width: 100px;
            padding: 5px;
            margin-top: 10px;
        }
        a {
            display: inline-block;
            margin-top: 15px;
        }
    </style>
</head>
<body>
    <h1>회원 등록</h1>

    <form th:action="@{/members}" th:object="${member}" met
hod="post">
        <label>
            이름:
            <input type="text" th:field="*{name}" placehold
er="이름 입력"/>
        </label>
        <label>
            Email:
            <input type="email" th:field="*{email}" placeho
lder="Email 입력"/>
        </label>
        <button type="submit">등록</button>
    </form>

    <a href="/members">목록으로</a>
</body>
</html>
```

전체 아키텍처

1. Spring Boot → JPA, Thymeleaf로 앱 개발
2. MySQL 연동
3. Jenkins CI/CD에서 Secret File로 DB 정보 관리
4. DockerHub & Nginx로 배포 확장 가능